GastoFlow · Propuesta del proyecto final

Desarrollo de Interfaces (0488) · Actividad individual 1 · DAM 2 · Nebrija · Jorge González Gómez · Octubre 2026

Repositorio: `github.com/JorgeGonzalezDI/gastoflow` · Producción: Vercel (enlace en la entrega final)

1. Qué es GastoFlow

GastoFlow es una aplicación de **gestión de gastos corporativos** (PWA/híbrida). Los empleados registran sus gastos de empresa (desplazamientos, dietas, material, formación...) con importe, categoría, fecha y descripción. El departamento de administración los revisa, los aprueba o rechaza, y consulta un **cuadro de mando** con el gasto por categoría, departamento y mes, exportable a **PDF y CSV**.

Por qué este tema: es un proceso real de cualquier empresa; los datos (importes, estados, categorías) dan sentido a los gráficos y a los informes; el flujo de aprobación justifica de forma natural los dos roles; y el tiempo real tiene un uso claro: el empleado ve al instante cuándo le aprueban o rechazan un gasto. Todos los datos serán sintéticos (empleados y empresas ficticias).

2. Roles y permisos (RBAC)

Rol	Quién es	Qué puede hacer
Administrador (`admin`)	Departamento de administración / finanzas	Todo lo del empleado + bandeja de aprobaciones, cuadro de mando con gráficos, gestión de usuarios y categorías, exportar informes PDF/CSV.
Usuario estándar (`empleado`)	Cualquier empleado de la empresa	Inicio, sus propios gastos (crear y editar mientras estén pendientes), ver el estado en tiempo real y la ayuda.

Cada rol tiene una lista de **permisos** (p. ej. `gastos:aprobar`, `informes:exportar`). Las rutas, el menú y los botones comprueban el permiso, no un simple `isAdmin`, así que añadir un rol nuevo (p. ej. supervisor) no obliga a tocar las pantallas.

3. Tecnologías

Capa	Tecnología	Motivo
Interfaz	Ionic 9 + Angular 22 (standalone, signals, sin zone.js)	Lo trabajado en el curso; una sola base para web, Android e iOS (Capacitor).
Datos y sesión	Supabase: Postgres (SQL), Auth (JWT), Realtime y API REST	Indicado por el profesor. Cubre SQL, autenticación con roles y la parte en tiempo real (columnas `jsonb` para datos tipo documento).
Gráficos e informes	Chart.js · jsPDF + autotable · CSV nativo	Librerías ligeras y estándar para el cuadro de mando y la exportación.
Calidad	Lighthouse · Compodoc · ESLint · Vitest	Auditoría de accesibilidad (>= 90), documentación generada y tests.
Entrega	Git + GitHub (Git Flow) · Vercel · GitHub Actions	Historial revisable, despliegue automático desde `main` y CI básica.

4. Arquitectura y estructura

Patrón **MVVM**: el componente es la vista (solo pinta), el servicio con signals es el *ViewModel* (estado y lógica) y Supabase es el modelo. La seguridad (guards, interceptor HTTP y RBAC) está documentada en `docs/security-architecture.md`.

```
src/app/
  core/       una sola instancia en toda la app: auth (servicio, guards), modelos, interceptor
  shared/     reutilizable: componentes (tabla con buscador, widget estadístico), directivas
  layout/     shell con menú lateral (ion-split-pane + ion-menu) filtrado por permisos
  features/   una carpeta por pantalla, todas con carga diferida (loadComponent)
              auth/login · inicio · gastos · admin/aprobaciones · admin/dashboard · ayuda
docs/        propuesta, memoria de seguridad, guía del código
```

5. Modelo de datos (borrador)

- `perfiles` (id = usuario de Supabase Auth, nombre, email, departamento, rol).
- `categorias` (id, nombre, límite mensual).
- `gastos` (id, empleado_id, categoria_id, fecha, importe, descripción, estado: pendiente / aprobado / rechazado, `detalles jsonb`).
- Seguridad en la base de datos con políticas RLS: un empleado solo puede leer y modificar sus gastos; el administrador, todos.

6. Plan por fases

Fase (RA · peso)	Contenido	Estado
1 · Estructura y formularios (RA 1-2 · 35%)	Menú lateral, lazy loading, guards por rol (canMatch), login con formulario reactivo; alta de gastos y registro con validación asíncrona (email ya existente).	En curso
2 · Componentes (RA 3 · 15%)	Tabla de datos con buscador y widget estadístico, con inputs/outputs y sin depender de la vista.	Pendiente
3 · Datos e informes (RA 5 · 15%)	Servicios async (REST + Realtime), cuadro de mando con Chart.js, exportación PDF/CSV, spinners y errores.	Pendiente
4 · Accesibilidad (RA 4 · 15%)	Mobile-first, contraste, teclado, ARIA; Lighthouse >= 90 en Accessibility y Best Practices.	Desde el inicio
5 · Documentación y despliegue (RA 6-7 · 20%)	Compodoc, manual de usuario (README + pantalla de ayuda), Git Flow con Pull Requests, CI y Vercel.	En curso

7. Git y ramas

- `main`: solo lo terminado. Vercel publica automáticamente desde esta rama.
- `develop`: integración del trabajo en curso.
- `feature/*`: una rama por funcionalidad (p. ej. `feature/estructura-layout`, `feature/auth-guards`). Sale de develop y vuelve a ella con merge/Pull Request.
- Commits pequeños con mensajes del tipo `feat:`, `fix:`, `docs:`. Al cerrar cada fase, develop pasa a main con una versión (`v0.1`, `v0.2`...).

8. Entrega final

Un .zip `GONZALEZ_GOMEZ_JORGE_DI_PROYECTO.zip` con un PDF que incluya: URL del repositorio, URL de producción, credenciales de prueba de Administrador y Empleado (solo en el PDF, nunca en el repositorio), auditoría Lighthouse y enlace a la documentación generada y al manual de usuario.